

CampusNavigation 代码骨架说明

这份文档介绍 `CampusNavigation/` 文件夹里每一个 `.h` 和 `.cpp` 文件是干什么的

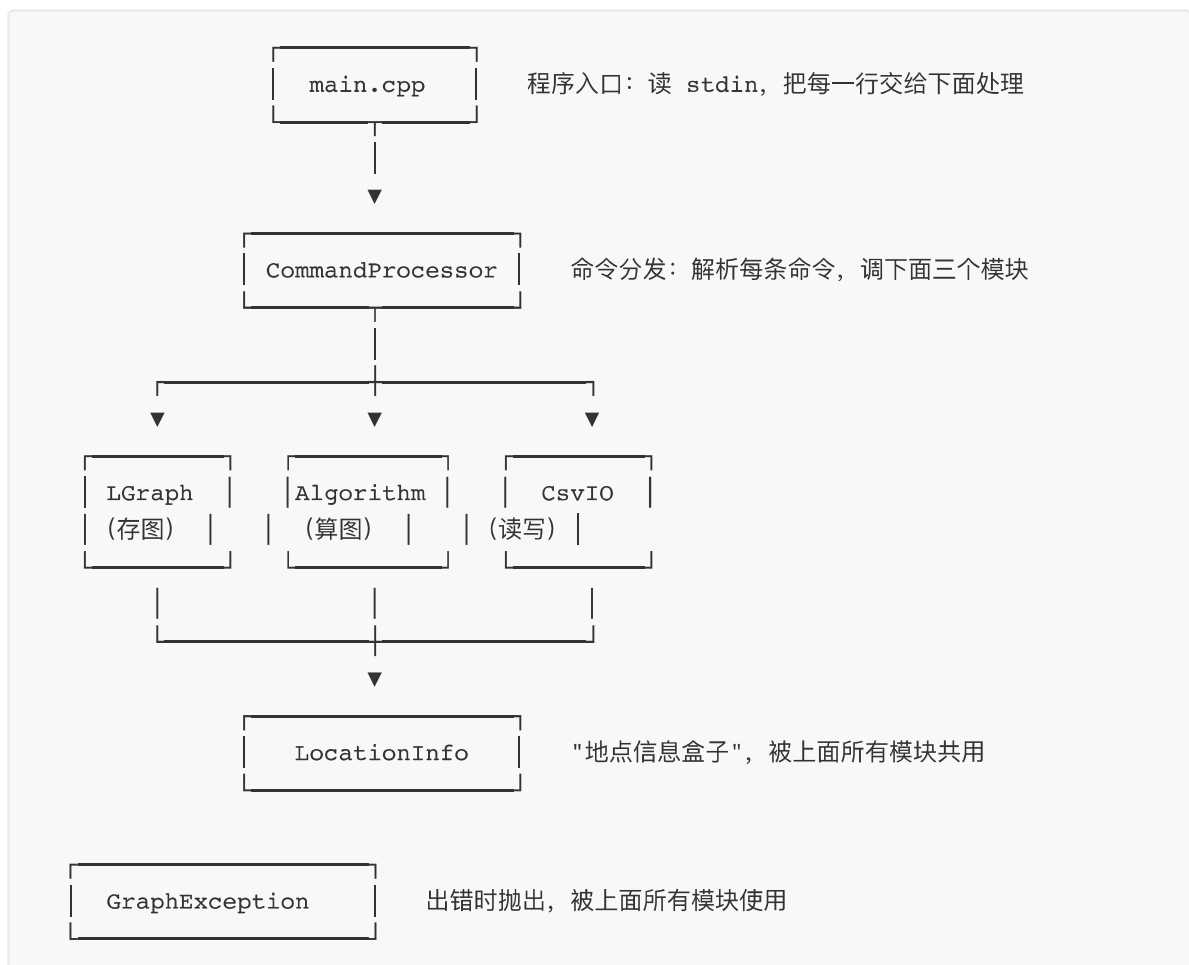
0. 这个文件夹是什么

`CampusNavigation/` 是你的代码工作区。整个项目要求你实现一个"动态校园图"的命令行程序：

- 程序从 **标准输入** (stdin) 一行一行读命令 (比如 `LOAD places.csv roads.csv`、`SHORTEST P0001 P0010 DIST`)
- 内部维护一张图, 包含校园里的地点和道路
- 每读一行命令就执行对应操作, 把结果按规范格式打印到 **标准输出** (stdout)

1. 模块关系总览

1.1 模块关系图



1.2 文件一览表

文件	作用
<code>main.cpp</code>	程序入口，读 <code>stdin</code>
<code>CMakeLists.txt</code>	构建配置
<code>LocationInfo.h</code>	地点信息结构体
<code>LocationInfo.cpp</code>	(空实现，全部内联)
<code>GraphException.h</code>	异常类
<code>LGraph.h</code>	图 ADT 公共接口声明
<code>LGraph.cpp</code>	图 ADT 实现
<code>Algorithm.h</code>	图算法接口声明
<code>Algorithm.cpp</code>	图算法实现
<code>CsvIO.h</code>	CSV 读写接口声明
<code>CsvIO.cpp</code>	CSV 读写实现
<code>CommandProcessor.h</code>	命令分发器声明
<code>CommandProcessor.cpp</code>	命令分发器实现

2. 每个文件详解

2.1 LocationInfo

文件：`LocationInfo.h`、`LocationInfo.cpp`

```
struct LocationInfo {
    std::string place_id;           // 唯一 ID, 如 P0001
    std::string display_name;       // 展示名称
    std::string category;           // 类别: Teaching/Dining/Dormitory/...
    int stay_time;                  // 建议停留时间 (分钟)
    std::string open_time;          // 开放时间, 如 "08:00"
    std::string close_time;         // 关闭时间, 如 "22:00"
};
```

`CsvIO::ReadPlaces` 返回 `vector<LocationInfo>`, `LGraph::GetVertex` 也返回它。它是各模块之间传递"地点信息"的通用载体。

2.2 GraphException

文件：`GraphException.h`

自定义异常类，遇到错误时 `throw` 它。

```
class GraphException : public std::exception {
    explicit GraphException(std::string msg);
    const char* what() const noexcept override;
};
```

```
if (!exist_vertex(id)) {
    throw GraphException("place not found: " + id);
}
```

2.3 LGraph

文件：LGraph.h、LGraph.cpp

图的 ADT——管理所有地点和道路，对外提供"增删改查"接口。

2.3.1 LGraph.h 结构

Part 1: private: 区——完全空白，等你自己设计存储。

```
class LGraph {
private:
    // =====
    // TODO: 在此处自行设计你的图数据结构
    // =====
    // 可选方案:
    // 1. 邻接表 (vector + list 等)
    // 2. 邻接矩阵
    // 3. 边表 + id→索引 map
    // 4. 嵌套 hash map
    // ...
};
```

你要在报告里说明你选的方案、考虑过哪些替代、复杂度分析。

Part 2: public:

类别	方法
基础信息	VertexCount、EdgesCount、exist_vertex、exist_edge
顶点操作	InsertVertex、DeleteVertex、UpdateVertex、GetVertex
边操作	InsertEdge、DeleteEdge、UpdateEdge、GetEdge
道路开关	CloseRoad、OpenRoad
遍历/查询	AllPlaceIds、AllEdges、GetAdjacentEdges、GetPlacesByCategory

另外还有一个辅助结构：

```

struct EdgeNode {
    std::string from_id, to_id;
    int distance, walk_time;
    std::string status;          // "open" 或 "closed"
};

```

图是无向的——`InsertEdge(A, B)` 在内部要让 A 和 B 互相能访问到对方。但 `AllEdges()` 返回时每条边只出现一次。

2.4 Algorithm（图算法）

文件：`Algorithm.h`、`Algorithm.cpp`

一句话定位：图算法模块——`LGraph` 负责"存"，`Algorithm` 负责"算"，干净分工。

2.4.1 `Algorithm.h` 提供了什么

算法函数（5 个）：

函数	算法	提示
<code>GetConnectedComponents</code>	连通分量	BFS 或 DFS
<code>GetShortestPath</code>	最短路径（DIST/TIME 双模式）	Dijkstra
<code>GetTimedShortestPath</code>	时刻约束最短路径	Dijkstra 变种，加时段过滤
<code>GetMustPassPath</code>	必经点路径	拆成若干段 Dijkstra 拼起来
<code>MinimumSpanningTree</code>	最小生成树	Kruskal 或 Prim 任选
<code>FindCriticalNodesAndEdges</code>	关键节点/关键边	暴力法：删一个看连通分量数变化

辅助结构（3 个）：

```

struct PathResult {
    int total_cost;                // 总代价（距离或时间）
    std::vector<std::string> path; // 路径上的 place_id 序列
    bool reachable;               // 是否可达
};

struct ComponentsResult {
    int count;                    // 连通分量个数
    std::vector<int> sizes;       // 各分量规模（降序）
};

struct CriticalResult {
    std::vector<std::string> critical_nodes;
    std::vector<std::pair<std::string, std::string>> critical_edges;
};

```

并查集类 (DSU) : 放在 `Algorithm.h` 里, 供 Kruskal 用, 自带 4 个 TODO:

```
class DSU {
    explicit DSU(int n);    // TODO: 初始化
    int find(int x);        // TODO: 路径压缩
    void unite(int x, int y); // TODO: 按秩合并
    bool same(int x, int y); // TODO
};
```

2.4.2 `Algorithm.cpp` 工作量

提示:

- 所有算法只考虑 `status="open"` 的边
- 时刻约束算法要做字符串比较 (`open_time <= time <= close_time`), 因为时间格式是 `HH:MM`, 字典序就是时间序
- 关键节点/边的暴力法复杂度 $O(V \cdot (V+E))$, 对 1000 节点规模够用

2.5 CsvIO (文件读写)

文件: `CsvIO.h`、`CsvIO.cpp`

一句话定位: CSV 文件管家——负责把 `places.csv`、`roads.csv` 读进来, 再能写回去。

2.5.1 接口 (4 个函数)

```
namespace CsvIO {
    std::vector<LocationInfo> ReadPlaces(const std::string &path);
    std::vector<RoadRecord> ReadRoads(const std::string &path);
    void WritePlaces(const std::string &path, const LGraph &graph);
    void WriteRoads(const std::string &path, const LGraph &graph);
}
```

`RoadRecord` 是道路的原始记录格式:

```
struct RoadRecord {
    std::string from_id, to_id;
    int distance, walk_time;
    std::string status;
};
```

提示:

- 兼容带表头和不带表头两种格式——如果第一行第一个字段是 `place_id` 或 `from_id`, 跳过这一行
- CSV 字段用逗号分隔, 不要假设有空格——`"P0001,Home"` 和 `"P0001, Home"` 是不同的
- 写回时要带表头 (参考 `places.csv` / `roads.csv` 第一行)

2.6 CommandProcessor（命令分发）

文件： `CommandProcessor.h`、`CommandProcessor.cpp`

一句话定位：命令翻译官——把每条命令拆开，调用对应的 `LGraph` / `Algorithm` / `CsvIO`，再按规范格式输出结果。

2.6.1 工作流程



2.6.2 19 个命令处理函数

命令类型	函数
文件 I/O	<code>cmdLoad</code> 、 <code>cmdSave</code>
查询	<code>cmdQueryPlace</code> 、 <code>cmdQueryCategory</code> 、 <code>cmdAdj</code>
增改地点	<code>cmdAddPlace</code> 、 <code>cmdDeletePlace</code> 、 <code>cmdUpdatePlace</code>
增改道路	<code>cmdAddRoad</code> 、 <code>cmdDeleteRoad</code> 、 <code>cmdUpdateRoad</code>
道路开关	<code>cmdCloseRoad</code> 、 <code>cmdOpenRoad</code>
图算法	<code>cmdComponents</code> 、 <code>cmdShortest</code> 、 <code>cmdTimedShortest</code> 、 <code>cmdMustPass</code> 、 <code>cmdMst</code> 、 <code>cmdCritical</code>

每个都和《命令接口规范.md》里的某条命令一一对应。

关键提示：

- 输出格式必须和参考答案一模一样——多一个空格、少一个 `ok`、换行错位都会被判错
- 出错时按规范输出 `ERROR ...`（用 `try/catch` 接 `GraphException`）
- 每条命令处理完通常要换行（`std::endl` 或 `"\n"`）

2.7 main.cpp

文件： `main.cpp`

一句话定位：入口——10 行有效代码，建图、建分发器、循环读 `stdin`。

```
int main() {
    Graph::LGraph graph(false);           // 无向图
    Graph::CommandProcessor processor(graph);

    std::string line;
    while (std::getline(std::cin, line)) {
        if (!processor.ProcessCommand(line)) {
            break;           // QUIT 命令返回 false
        }
    }
    return 0;
}
```

3. 编译 & 运行

3.1 最简单的编译命令

```
cd ~/Desktop/CampusDemo/CampusNavigation
clang++ -std=c++17 -Wall *.cpp -o CampusNavigation
```

Linux 同学把 `clang++` 换成 `g++`。

Windows 同学用 MinGW 或 MSVC。

3.2 用 CMake 不强求，可以用G++

```
cd ~/Desktop/CampusDemo/CampusNavigation
mkdir -p build && cd build
cmake ..
make
```

3.3 跑评测

```
# 假设你用方式 3.1 编译，当前还在 CampusNavigation/ 里
cp ../testcases/required/sample_ecnu/*.csv .
cp ../testcases/required/sample_ecnu/command.txt .

./CampusNavigation < command.txt > my_output.txt
diff my_output.txt ../testcases/required/sample_ecnu/answer.txt
```

`diff` 没有输出 = 你的程序和参考答案完全一致 = 通过。

4. 常见疑问

Q1: 可以不用这个骨架，全部自己重写吗？

A: 可以。但 CLI 接口（命令格式、输出格式）必须严格按《命令接口规范.md》实现，否则评测过不了。

Q2: LGraph 的 private 区为什么是空的?

A: 故意的——存图方式是数据结构课的核心训练点。你要自己设计、自己分析复杂度、在报告里说清楚。

Q3: 改了 .cpp 文件, 重新跑发现没变化?

A: 99% 是忘了重新编译。改完代码必须再跑一次 `clang++ -std=c++17 -Wall *.cpp -o CampusNavigation`。

Q4: 编译出一堆 warning, 能用吗?

A: warning 不会让编译失败, 但建议都修掉——很多 warning 其实是潜在 bug。error 才是真失败。

Q5: 程序跑起来一动不动?

A: 你大概率忘了用 `<` 重定向。`./CampusNavigation command.txt` ❌ 是把 command.txt 当参数; `./CampusNavigation < command.txt` ✅ 才是把它当输入。

Q6: CSV 里有中文怎么办?

A: 本项目数据全是英文, 不需要处理中文。如果你想扩展, 编码统一用 UTF-8。